EE2703 - Applied Programming Lab (Report 4)

Nithin G - EE24B046

1. Objective

The objective of the assignment is to read a digital circuit description (netlist) containing inputs, outputs, logic gates, and stimulus data and compute the output values by evaluating the logic gates in the correct dependency order.

2. Solving the Problem

Procedure to Solving the Problem

- The netlist was considered to be split into 4 sections: `INPUTS, OUTPUTS, GATES, STIMULUS`.
- The netlist was split into lines and then only the sections were taken out of it, further split into a list containing the lines of the `.net` file `Lines 33-35`
- The sections were identified from the list of lines by a function `expect()` which takes in inputs as the **section name** and **index no.** `Lines 37-41`
- The **inputs** line will have to be split w.r.t **":"** and the variables to the right of the **":"** are split and stored in a list as `inputs`. An error message is also implemented otherwise. `Lines 43-52`
- The **outputs** line will have to be split w.r.t **":"** and the variables to the right of the **":"** are split and stored in a list as `outputs`. An error message is also implemented otherwise. `Lines 54-63`
- The **gates** section is not contained in a line, and hence it wont be there as a single element in the `lines` list. The iteration of all lines under the **gates** section has to be done till the **stimulus** section. `out, typ, args` respectively store the output name of the variable, the gate used, the inputs to the gates and they are stored in as a tuple as `gates`. An error message is also implemented otherwise. `Lines 66-79`
- An error message is also created to reject unknown gates and mismatched input counts. `Lines 82-88`
- The **stimulus** section is not contained in a line, and hence it wont be there as a single element in the `lines` list. The iteration of all lines under the **stimulus** section has to be done till the end of the list. The **stimulus** section is seperated away from the first columns which is stored in a list called `times` and also the other coloumns are split and stored into the list called `stimulus`. An error message is also implemented otherwise. `Lines 92-102`
- An error message is also created to reject unsorted timestamps and duplicate time stamps. `Lines 105-109`
- The `simulate` function is now implemented. To encounter the problem of unordered gates under the gates section, it could be tackled by creating a duplicate list like `remaining` which iterates throughout all the elements of the list `gates`. If an input variable for a gate is not recognized then , that element

goes into the `next round` list and get computed once the first set of **remaining** gates were executed. This goes on till all the gates have been executed. An error message is also implemented otherwise. `Lines 131-158`

- The `to_wavedrom_json` function is implemented to output the whole outputs as in the required **json** format. Since I wan't knowing how json formats would look like , I had to GPT and learn the json format and look up on how to write this part. `Lines 161-173`
- The program is computed therefore after the defaultly given **main** function.

3. Analysis

An output of a sample test case is as follows:

Running : The `py digitalsim.py examples/complex_new.net -o complex_new.json`

Where the file **complex_new.net** is as follows:

```
INPUTS: A B C D E F G H
OUTPUTS: Y
GATES:
  T1 = AND(A, B)
  T2 = XOR(C, D)
  T3 = OR(T1, E)
  T4 = AND(T2, F)
  T5 = XOR(T3, T4)
  T6 = NOT(G)
  Y  = OR(T7, H)
  T7 = AND(T5, T6)

STIMULUS:
  0  0 0 0 0 0 0 0 0
  1  1 0 0 1 0 1 1 0
  2  1 1 1 0 1 0 0 0
  3  0 1 1 1 1 1 1 1
  4  1 1 0 1 0 1 0 1
  5  0 0 1 0 1 0 1 0
```

Note that this has got an unordered gate topology, where the **T7** was taken as an input before it was defined as the output.

The output in a **json** format:


```
{
  "signal": [
    {"name": "A", "wave": "011010"},
    {"name": "B", "wave": "001110"},
    {"name": "C", "wave": "001101"},
    {"name": "D", "wave": "010110"},
    {"name": "E", "wave": "001101"},
    {"name": "F", "wave": "010110"},
    {"name": "G", "wave": "010101"},
    {"name": "H", "wave": "000110"},
    {"name": "Y", "wave": "001110"}
  ]
}
```


The takeaway here is to note that the above json file shows the same output as when the netfile follows the correct topological order. This is in accordance to what the assignment asks to do.

Apart from the source code `complex_new.net` and `complex_new.json` have been added for reference.